DeepCompile

CLI Command Reference

Week 8 Edition · Analysis · Optimization · Verification · Security

Complete reference for every command you can run against a C/C++ source file.

Updated: Week 8 — Security Agent added, Phase 2 Integration Report generator added.

Full 4-Agent Pipeline — `run_pipeline.py`

Runs **Analysis** → **Optimization** → **Verification** → **Security** end-to-end.

```
python run_pipeline.py <file>
```

Options

Flag	Short	Effect
`--verbose`	`-v`	Show every step in DEBUG detail (LLM prompts, regex hits)
`--quiet`	`-q`	Show warnings and errors only
`--no-llm`		Skip Ollama — use rule-based analysis only (faster, offline)
`--no-security`		Skip Step 4 Security Agent (run only 3 stages)
`--show-messages`	`-m`	Print live inter-agent messages on stdout
`--save-messages FILE`		Save inter-agent messages as JSON to FILE

Pipeline Stages

```
Step 1/4 Analysis Agent — detects bugs, inefficiencies, security issues
Step 2/4 Optimization Agent — applies rule-based and LLM-driven fixes
Step 3/4 Verification Agent — differential test + Z3 SMT + perf benchmark + LLM
Step 4/4 Security Agent — 4-layer vulnerability audit on optimized code
```

Examples

```
# Default (INFO level logging)
python run_pipeline.py MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp

# Verbose — see every LLM prompt and regex hit
python run_pipeline.py MicroBenchmarks/Testcases/TC11_buffer_overflow_loop.cpp --verbose

# Quiet — only errors/warnings
python run_pipeline.py MicroBenchmarks/Testcases/TC07_div_by_zero_var.cpp --quiet

# Skip LLM (much faster, no Ollama needed)
python run_pipeline.py MicroBenchmarks/Testcases/TC05_uninit_struct.cpp --no-llm

# Skip Security Agent (3-stage pipeline only)
python run_pipeline.py MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp --no-security

# Skip both LLM and Security (fastest, pure rule-based)
```

```
python run_pipeline.py MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp --no-llm
--no-security

# Show inter-agent messages live
python run_pipeline.py MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp --show-messages

# Save messages to a JSON file
python run_pipeline.py MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp --save-messages
logs/messages.json

# Combine flags
python run_pipeline.py MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp --no-llm --verbose
```

Pipeline Result Status Values

Status	Meaning
`success`	All 4 stages passed — optimized file saved
`rollback`	Verification or Security found a critical issue — original code preserved
`partial`	Verification FAIL but not a rollback — output may be unreliable
`failed`	Pipeline could not complete (file not found, fatal error)

Security Rollback Logic

The Security Agent (Step 4) triggers a rollback **only** if:

- A **new** HIGH-severity vulnerability was introduced by optimization (not present in original)
- The finding comes from **rule-based** or **heuristic** detection (confidence ≥ 0.8)
- LLM-only findings never trigger rollback

Single-Agent Commands — `run_agent.py`

Run exactly **one** agent without the others.

```
python run_agent.py <command> <file> [options]
```

Sub-commands

Sub-command	Agents run	Output
`analyze`	Analysis only	Findings list with severity, line numbers
`optimize`	Analysis → Optimization	Diff + saved `OPT_<file>`
`verify`	Verification only	4-layer verification report

2a · Analyze only

```
python run_agent.py analyze <file> [--no-llm] [-v|-q]
```

```
# Basic analysis
python run_agent.py analyze MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp

# Rule-based only, no Ollama
python run_agent.py analyze MicroBenchmarks/Testcases/TC11_buffer_overflow_loop.cpp --no-llm
```

```
# Verbose output
python run_agent.py analyze MicroBenchmarks/Testcases/TC05_uninit_struct.cpp --verbose

# Quiet (errors only)
python run_agent.py analyze MicroBenchmarks/Testcases/TC07_div_by_zero_var.cpp --quiet
```

2b · Optimize only

Runs a quick Analysis internally first (to collect context), then applies optimizations.

No verification or security pass.

```
python run_agent.py optimize <file> [--no-llm] [-v|-q]
```

```
# Optimize with LLM
python run_agent.py optimize MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp

# Optimize offline (rule-based fixes only)
python run_agent.py optimize MicroBenchmarks/Testcases/TC11_buffer_overflow_loop.cpp --no-llm

# Verbose
python run_agent.py optimize MicroBenchmarks/Testcases/TC07_div_by_zero_var.cpp --verbose
```

Optimized file is saved to:

```
MicroBenchmarks/Generated_optimisation/OPT_<original_filename>.cpp
```

2c · Verify only

Runs the 4-layer verification (Differential Testing → Z3 SMT → Performance Benchmark → LLM Reasoning).

Needs an already-optimized file.

```
python run_agent.py verify <file> [--optimized <opt_file>] [-v|-q]
```

```
# Auto-detect optimized file from Generated_optimisation/
python run_agent.py verify MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp

# Point to a specific optimized file
python run_agent.py verify MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp \
--optimized MicroBenchmarks/Generated_optimisation/OPT_TC01_uninit_arithmetic.cpp

# Verbose verification (see each layer's detail)
python run_agent.py verify MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp \
--optimized MicroBenchmarks/Generated_optimisation/OPT_TC01_uninit_arithmetic.cpp \
--verbose
```

*Note: **Auto-detect rule:** if `--optimized` is omitted, looks for `MicroBenchmarks/Generated_optimisation/OPT_.cpp`*

Optimizer-Only Shortcut — `run_optimizer.py`

Focused entry point: **Analysis** → **Optimization**, no verification or security.

Shorter output than `run_agent.py optimize`, supports custom output path.

```
python run_optimizer.py <file> [--no-llm] [--out <path>] [-v|-q]
```

Options

Flag	Effect
`--no-llm`	Rule-based fixes only
`--out <path>`	Custom path for the optimized file
`--verbose` / `-v`	DEBUG logging
`--quiet` / `-q`	WARNING-only logging

Examples

```
# Basic usage
python run_optimizer.py MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp

# Rule-based only (no Ollama)
python run_optimizer.py MicroBenchmarks/Testcases/TC11_buffer_overflow_loop.cpp --no-llm

# Save to custom path
python run_optimizer.py MicroBenchmarks/Testcases/TC07_div_by_zero_var.cpp \
--out MicroBenchmarks/Generated_optimisation/my_fixed.cpp

# Quiet with custom output
python run_optimizer.py MicroBenchmarks/Testcases/TC05_uninit_struct.cpp \
--no-llm --out results/TC05_fixed.cpp --quiet
```

Message Watcher — `watch_agents.py`

Like `run_pipeline.py` but streams **every inter-agent message** to the terminal in real time, with a final summary table.

```
python watch_agents.py <file> [--full] [--no-llm]
```

Options

Flag	Effect
`--full`	Show complete payloads including full source code and diffs
`--no-llm`	Skip Ollama

Examples

```
# Watch all messages (payloads truncated by default)
python watch_agents.py MicroBenchmarks/Testcases/TC11_buffer_overflow_loop.cpp

# Show full payloads (including source code)
python watch_agents.py MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp --full

# Rule-based mode
python watch_agents.py MicroBenchmarks/Testcases/TC05_uninit_struct.cpp --no-llm
```

Phase 2 Integration Report — `evaluation/generate_report.py`

Runs the full 4-stage pipeline on a representative set of test cases and generates a **Markdown report** summarising per-agent performance, security findings, and overall pass rates.

```
python evaluation/generate_report.py
```

Options

Flag	Effect
`--no-llm`	Disable LLM for all agents (rule-based only, much faster)
`--out <path>`	Custom output path for the Markdown report
`--cases TC01 TC11 ...`	Filter to specific test cases by filename prefix

Examples

```
# Full report (LLM enabled — takes several minutes)
python evaluation/generate_report.py

# Fast offline report (rules only)
python evaluation/generate_report.py --no-llm

# Custom output path
python evaluation/generate_report.py --no-llm --out reports/my_report.md

# Run only specific test cases
python evaluation/generate_report.py --no-llm --cases TC01 TC11 TC16

# Single test case report
python evaluation/generate_report.py --no-llm --cases TC19
```

Default Test Cases

The report runs on 8 representative cases spanning all major bug categories:

Test Case	Category
TC01	Uninitialized variable — arithmetic
TC06	Division by zero — simple
TC09	Array out of bounds
TC11	Buffer overflow — loop
TC12	strcpy buffer overflow
TC14	Memory leak
TC16	Use-after-free
TC19	Heap overflow

Default Report Output

```
Technical Deliverlables and Documents/Phase2_Integration_Report.md
```

Running Tests — `pytest`

```
# All tests
python -m pytest tests/ -v

# Week 8 security tests only
python -m pytest tests/test_security_week8.py -v

# Specific test class
```

```
python -m pytest tests/test_security_week8.py::TestRuleBasedDetection -v

# All weeks, quiet summary
python -m pytest tests/ -q

# With coverage report
python -m pytest tests/ --cov=src --cov-report=term-missing
```

Test Files

File	Covers
`test_llm_and_parser.py`	LLM client, prompt templates, code parser
`test_agents_week6.py`	CoT validator, Analysis Agent, Optimization Agent
`test_verification_week7.py`	Diff tester, Z3 verifier, perf benchmarker, Verification Agent
`test_security_week8.py`	Security Agent (all 4 layers), rollback logic, pipeline integration

---

Command Comparison Table

Goal	Command
Full 4-agent audit	`python run_pipeline.py <file>`
Full audit + see agent messages	`python run_pipeline.py <file> --show-messages`
Full audit, no LLM	`python run_pipeline.py <file> --no-llm`
Full audit, skip security	`python run_pipeline.py <file> --no-security`
Fastest possible scan	`python run_pipeline.py <file> --no-llm --no-security`
Stream all messages live	`python watch_agents.py <file>`
Just detect bugs/patterns	`python run_agent.py analyze <file>`
Fix code, skip verification	`python run_agent.py optimize <file>`
Verify an existing optimized file	`python run_agent.py verify <file> --optimized <opt>`
Quick optimize, focused output	`python run_optimizer.py <file>`
Optimize offline (no Ollama)	`python run_optimizer.py <file> --no-llm`
Batch report on test suite	`python evaluation/generate_report.py --no-llm`

LLM Modes

All commands support `--no-llm` to skip Ollama entirely and use rule-based logic only.

LLM Mode	When to use	Speed
(default) — LLM + rules	Ollama is running (`ollama serve`)	Slower, richer output
`--no-llm` — rules only	No Ollama / quick offline scan	Fast

To start Ollama: `ollama serve` (in a separate terminal).

Model used: **Qwen 2.5 Coder 7B** (pulled with `ollama pull qwen2.5-coder:7b`).

Verbosity Levels

All commands support the same logging flags:

Flag	Level	Shows
`-v` / `--verbose`	DEBUG	LLM prompts, regex hits, every internal step
(default)	INFO	One line per pipeline step — recommended
`-q` / `--quiet`	WARNING	Only warnings and errors

Log file (always DEBUG, regardless of console setting):

```
logs/pipeline.log
```

Security Agent Detection Layers

The Security Agent (Step 4) runs four detection layers in order:

Layer	Method	Confidence	Can trigger rollback?
1 — Rule-based	Exact regex patterns for unsafe C/C++ functions	0.90	Yes (if new + HIGH)
2 — Heuristic	Multi-signal context-aware scoring (taint flow, unchecked malloc, etc.)	0.65–0.85	Yes (if score ≥ 0.8 + new + HIGH)
3 — LLM	Semantic analysis via SecurityPromptTemplate	0.70	No (informational only)
4 — cppcheck	Static analysis tool XML output (skipped if not installed)	0.85	No (informational only)

Vulnerability Types Detected

Type	CWE	Layer
`unsafe_gets`	CWE-242	Rule
`unsafe_scanf_s`	CWE-134	Rule
`unsafe_strcpy`	CWE-676	Rule
`unsafe_strcat`	CWE-676	Rule
`unsafe_sprintf`	CWE-676	Rule
`format_string_bug`	CWE-134	Rule
`use_after_free`	CWE-416	Rule (C free + C++ delete)
`taint_flow_risk`	CWE-120	Heuristic
`unchecked_malloc`	CWE-476	Heuristic
`signed_unsigned_mismatch`	CWE-195	Heuristic
`integer_overflow_malloc`	CWE-190	Heuristic
`unbounded_loop_write`	CWE-121	Heuristic
`stack_overflow_risk`	CWE-674	Heuristic
`race_condition_risk`	CWE-362	Heuristic

File Path Notes

- Paths can be **relative** (from `E:\SEM 4\CD_Refined\`) or **absolute**.
- Test cases are in `MicroBenchmarks/Testcases/` — filenames follow `TC_.cpp`.
- Optimized outputs are saved to `MicroBenchmarks/Generated_optimisation/OPT_.cpp` unless overridden with `--out`.
- Message logs (if using `--save-messages`) default to `logs/messages.json`.
- The integration report is saved to `Technical Deliverlables and Documents/Phase2_Integration_Report.md`.

```
# Relative path (most common)
python run_pipeline.py MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp

# Absolute path
python run_pipeline.py "E:\SEM
4\CD_Refined\MicroBenchmarks\Testcases\TC01_uninit_arithmetic.cpp"
```

DeepCompile · 24CSB0A06 · Week 8